SEABORN

Seaborn is an amazing data visualization library for statistical graphics plotting in Python.

It provides beautiful default styles and colour palettes to make statistical plots more attractive.

It is built on the top of the matplotlib library and also closely integrated to the data structures from pandas.

Here is some of the functionality that seaborn offers:

- A dataset-oriented API for examining relationships between multiple variables
- Specialized support for using categorical variables to show observations or aggregate statistics
- Options for visualizing univariate or bivariate distributions and for comparing them between subsets of data
- Automatic estimation and plotting of linear regression models for different kinds of dependent variables
- Convenient views onto the overall structure of complex datasets
- High-level abstractions for structuring multi-plot grids that let you easily build complex visualizations
- Concise control over matplotlib figure styling with several built-in themes
- Tools for choosing colour palettes that faithfully reveal patterns in your data

Difference Between Matplotlib and Seaborn

	MatPlotLib	Seaborn
Functionality	Matplotlib is mainly deployed for basic plotting. Visualization using Matplotlib generally consists of bars, pies, lines, scatter plots and so on.	Seaborn, on the other hand, provides a variety of visualization patterns. It uses fewer syntax and has easily interesting default themes. It specializes in statistics visualization and is used if one has to summarize data in visualizations and also show the distribution in the data.
Handling Multiple Figures	Matplotlib has multiple figures can be opened, but need to be closed explicitly. plt.close() only closes the current figure. plt.close('all') would close em all.	Seaborn automates the creation of multiple figures. This sometimes leads to OOM (out of memory) issues.
Visualization	Matplotlib is a graphics package for data visualization in Python. It is well	Seaborn is more integrated for working with Pandas data frames. It extends the Matplotlib

SEABORN

	integrated with NumPy and Pandas. The pyplot module mirrors the MATLAB plotting commands closely. Hence, MATLAB users can easily transit to plotting with Python.	library for creating beautiful graphics with Python using a more straightforward set of methods.
Data Frames and Arrays	Matplotlib works with data frames and arrays. It has different stateful APIs for plotting. The figures and aces are represented by the object and therefore plot() like calls without parameters suffices, without having to manage parameters.	Seaborn works with the dataset as a whole and is much more intuitive than Matplotlib. For Seaborn, replot() is the entry API with 'kind' parameter to specify the type of plot which could be line, bar, or any of the other types. Seaborn is not stateful. Hence, plot() would require passing the object.
Flexibility	Matplotlib is highly customizable and powerful.	Seaborn avoids a ton of boilerplate by providing default themes which are commonly used.
Use Cases	Pandas uses Matplotlib. It is a neat wrapper around Matplotlib.	Seaborn is for more specific use cases. Also, it is Matplotlib under the hood. It is specially meant for statistical plotting.

Installing Seaborn

```
pip install seaborn

conda install seaborn
```

Load Data

```
import pandas
import matplotlib
import scipy
import seaborn as sns
```

there is no of dataset is available with seaborn

```
dt=sns.get_dataset_names()
dt
```

Import any one of those datasets and visualize the data

```
df = sns.load_dataset('car_crashes')
df.head()
```

SEABORN

Themes in Seaborn

People are more likely to concentrate on beautiful and attractive visualizations rather than dull plots thus styling can be considered as a vital component of data visualization.

Matplotlib library is highly customizable, but it may be hard for get an attractive and good looking plot.

Seaborn comes packed with customized themes and a high-level interface for customizing and controlling the look of Matplotlib figures.

simple Matplotlib plot

in the data car_crashes create a plot using **speeding and alcohol column**

```
plt.scatter(df.speeding,df.alcohol)
plt.show()
```

using **set()** function of seaborn see how this plot changing

```
import seaborn as sns
plt.scatter(df.speeding,df.alcohol)
sns.set()
plt.show()
```

Above two figures show the difference in the default Matplotlib and Seaborn plots.

The representation of data is the same, but there is a slight difference in the styling of these plots.

Seaborn supports various themes that can make styling the plots really easy and save a lot of time.

Using the set_style() function of Seaborn we can set any of the themes available on Seaborn library.

- Darkgrid
- Whitegrid
- Dark
- White
- Ticks

Apply this theme and check how are they differ of each other
default theme of the plot is Darkgrid

SEABORN

```
import matplotlib.pyplot as plt
import seaborn as sns
plt.scatter(df.speeding,df.alcohol)
sns.set_style("dark")
plt.show()
```

We can remove the top and right axis spines using the despine() function.

```
import matplotlib.pyplot as plt
import seaborn as sns
plt.scatter(df.speeding,df.alcohol)
sns.set_style("ticks")
sns.despine()
plt.show()
```

customize themes

check the parameter of theme of axis style

```
param=sns.axes_style()
param
```

using this parameter of axis style to check

```
from matplotlib import pyplot as plt
import seaborn as sns
plt.scatter(df.speeding,df.alcohol)
sns.set_style("darkgrid",{'grid.color': '.5'})
sns.despine()
plt.show()
```

Templates of our graphs

Seaborn also allows us to control individual elements of our graphs and thus we can control the scale of these elements or the plot by using the set_context() function.

four preset templates for contexts, based on relative size

- Paper
- Notebook
- Talk
- Poster

SEABORN

By default, context is set to notebook

```
from matplotlib import pyplot as plt
import seaborn as sns
plt.scatter(df.speeding,df.alcohol)
sns.set_style("dark")
sns.set_context("notebook")
plt.show()
```

change context

```
from matplotlib import pyplot as plt
import seaborn as sns
plt.scatter(df.speeding,df.alcohol)
sns.set_style("dark")
sns.set_context("poster")
plt.show()
```

Plotting with the relplot function

The Seaborn library provides us with relplot() function and this function provides access to several different axes-level functions that show the relationship between two variables with semantic mappings of subsets.

- scatterplot() (with kind=”scatter”)
- lineplot() (with kind=”line”)

The default value for the parameter kind is ‘scatter’ which means that by default this function would return a scatterplot.

```
import seaborn as sns
tip = sns.load_dataset('tips')
tip
```

now check plot

```
sns.relplot(x="total_bill", y="tip", data=tip,hue="smoker")
```

for different categorised based on style and size

```
sns.relplot(data=dt,x='total_bill',y='tip',hue=smoker,style="time",
style="size",col=’sex’))))
```

CREATIVE INSTITUTE DATA SCIENCE

SEABORN

Bar Plot

Seaborn supports many types of bar plots

```
sns.barplot(x='day', y='total_bill', data=tip)

sns.catplot(x='day', y='total_bill', data=tip)

sns.catplot(x='day', y='total_bill', data=tip,jitter=False)

sns.catplot(x='day', y='total_bill', data=tip,kind='swarm')

sns.catplot(x='day', y='total_bill', data=tip,kind='box')
```

Line Plot

```
import seaborn as sns
dt = sns.load_dataset("tips")
dt.head()
sns.relplot(data=dt,x='size',y='tip',kind='line',ci=None)
```

Histogram

Histograms represent the data distribution by forming bins along with the range of the data and then drawing bars to show the number of observations that fall in each bin.

```
import seaborn as sns
from matplotlib import pyplot as plt
df = sns.load_dataset('iris')
sns.distplot(df['petal_length'],kde = False)
```

CREATIVE INSTITUTE DATA SCIENCE

SEABORN

Count plot

The count plot can be thought of as a histogram across a categorical variable.

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_context('paper')

# load dataset
dt = sns.load_dataset('titanic')
# create plot
sns.countplot(x = 'class', hue = 'who', data = dt, palette = 'magma')
plt.title('Survivors')
plt.show()
```

Point Plot

Point plot is used to show point estimates and confidence intervals using scatter plot graphs.

point plot represents an estimate of central tendency for a numeric variable by the position of scatter plot points and provides some indication of the uncertainty around that estimate using error bars.

Point plots can be more useful than bar plots for focusing comparisons between different levels of one or more categorical variables.

```
import seaborn as sns
import matplotlib.pyplot as plt

# loading dataset
dt = sns.load_dataset("tips")
sns.pointplot(x="day", y="tip", data=dt)
plt.show()
```

comparision between two variable

```
import seaborn as sns
import matplotlib.pyplot as plt

# loading dataset
dt = sns.load_dataset("tips")
sns.pointplot(x="day", y="tip", data=dt,hue="smoker")
plt.show()
```

SEABORN

Joint Plot

Joint Plot draws a plot of two variables with bivariate and univariate graphs. It uses the Scatter Plot and Histogram.

Joint Plot can also display data using Kernel Density Estimate (KDE) and Hexagons.

```
import seaborn as sns
import matplotlib.pyplot as plt
sns.set_style("dark")
dt=sns.load_dataset('tips')
sns.jointplot(x='total_bill', y='tip',data=dt)
```

We can also draw a Regression Line in Scatter Plot.

```
sns.jointplot(x='total_bill', y='tip',data=dt,kind='reg')
```

kernal density estimate

```
import seaborn as sns
import matplotlib.pyplot as plt
sns.set_style("dark")
dt=sns.load_dataset('tips')
sns.jointplot(x='total_bill', y='tip',data=dt,kind='kde')
```

Regplot

Regplot function used to visualize the linear relationship as determined through regression.

```
import seaborn as sns
dt = sns.load_dataset("tips")
ax = sns.regplot(x="total_bill", y="tip", data=dt)
```

remove highlighted area

use ci=None

```
ax = sns.regplot(x="total_bill", y="tip", data=dt,ci=None)
```

other plot is

LmPlot

CREATIVE INSTITUTE DATA SCIENCE

SEABORN

KDE plot

Box plot

Violin plot

Heatmap

Cluster map

Facet Grid

Pair plot